DepPy: Sheaf-Descent Semantic Typing for Python

One framework for type checking, equivalence, and bug finding

Halley Young

Every program analysis does the same thing:

1. Collect **local facts** at program points (types, invariants, pre/post-conditions)
2. **Compose** them into a global conclusion (whole-program typing, correctness proof, bug report)
3. Report where composition **fails** (type errors, counterexamples, alarms)

The catch: every tool re-invents composition from scratch.

Abstract interpretation	fixed-point iteration + widening
Type checking	structural induction over typing rules
Hoare logic	weakest precondition / sequential composition
Model checking	reachability over product automaton

Key insight: Sheaf theory *is* the mathematics of local-to-global composition. One framework, one soundness proof, all analyses.

1. Sites — observation points in a program

A category **S** whose objects are program points (argument boundaries, branch guards, return sites, SSA values, ...) and morphisms are data-/control-flow edges. DepPy defines **14 site families** (`SiteKind`).

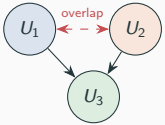
2. Sections — type facts at each site

A *presheaf* $\mathcal{F} : \mathbf{S}^{\mathrm{op}} \to \mathbf{Poset}$ assigns a poset of facts to each site, with *restriction maps* along edges. Each section carries: carrier type, refinements, trust level, provenance, witnesses.

3. Covers — when do sites "see everything"?

A *Grothendieck topology* J declares which site families $\{U_i \to U\}$ *cover* U, via three axioms:

- **Identity**: self-cover
- **Stability**: pullback preserves covers (specialization is free)
- **Transitivity**: refining covers compose (modular analysis for free)



Given a cover $\{U_i\}$ and a local section $\sigma_i \in \mathcal{F}(U_i)$ at each site:

$$\begin{tikzcd} \mathcal{F}(U) \arrow[r, "\text{restrict}"] & \prod_i \mathcal{F}(U_i) \arrow[r, shift left, "\mathrm{res}_1"] \arrow[r, shift right, "\mathrm{res}_2"'] & \prod_{i,j} \mathcal{F}(U_i \cap U_j) \end{tikzcd}$$

global *local* *overlaps*

Sheaf condition (implemented in `SheafConditionChecker`):

1. **Agreement**: on every overlap $U_i \cap U_j$, the restricted sections agree: $\sigma_i|_{U_i \cap U_j} = \sigma_j|_{U_i \cap U_j}$
2. **Gluing**: if all overlaps agree, there exists a *global section* $\sigma \in \mathcal{F}(U)$ that restricts to each σ_i
3. **Uniqueness**: the global section is unique

Key insight: Agreement $\Rightarrow$ sound composition. Failure to agree $\Rightarrow$ a *cocycle* $[\alpha] \in H^1$ that **names the exact pair of sites** where things go wrong.

Čech cohomology grades the obstruction:

Degree	Mathematical meaning	PL meaning
$H^0(\mathcal{U}, \mathcal{F})$	Global sections	Consistent whole-program typing
$H^1(\mathcal{U}, \mathcal{F})$	Gluing obstructions	**Type errors / bugs**
$H^2(\mathcal{U}, \mathcal{F})$	Higher obstructions	Higher-order inconsistencies

Traditional:

"Line 42: expected `int`, got `str`"

One location; root cause elsewhere.

Sheaf obstruction:

"Overlap of U_3, U_7: `int` vs `str`"

Names both sites + witness.

DepPy's `ObstructionData`: cohomology class + conflicting overlaps + explanation. Trivial H^1 = fix one section. Non-trivial = genuine conflict.

Standard PL	Sheaf-theoretic reading in DepPy
Abstract interp. lattice join	Gluing local sections at merge-point overlaps
Refinement subtyping	Restriction map between sites (contravariant functoriality)
Separation logic frame rule	Restriction along site morphisms; $*$ = disjoint site union
Liquid Haskell qualifier min.	Čech H^1 elimination of redundant VCs
Logical relations fund. thm	Stalk verification: η is iso iff η_p bijective $\forall p$
Bisimulation closure	Naturality squares of $\eta : \mathcal{F}_f \to \mathcal{F}_g$
IFDS/IDE summaries	Grothendieck transitivity axiom (modular composition)
Gradual typing boundaries	Trust-level annotations (PROOF-BACKED vs. ASSUMED)

Key insight: These aren't analogies — they're formal instances. Each PL technique is a specific choice of presheaf and topology.

Question: do two implementations f, g behave the same?

Sheaf approach: build presheaves $\mathcal{F}_f$, $\mathcal{F}_g$ and check for a natural isomorphism $\eta : \mathcal{F}_f \xrightarrow{\sim} \mathcal{F}_g$.

`GlobalEquivalenceChecker` runs **6 phases**:

1. **Local**: $\mathcal{F}_f(U_i) \cong \mathcal{F}_g(U_i)$ per site
2. **Naturality**: local isos commute with restrictions
3. **Descent**: $H^1(\mathcal{U}, \mathrm{Iso}(\mathcal{F}_f, \mathcal{F}_g)) = 0$?
4. **Gluing**: construct global equivalence via fiber product
5. **Uniqueness**: sheaf condition
6. **Stalk verification**: cross-check at all stalks

```
from deppy.equivalence.pipeline import EquivalencePipeline, EquivalencePipelineConfig
pipe = EquivalencePipeline(EquivalencePipelineConfig(function_name="sort"))
result = pipe.run("sort_v1.py", "sort_v2.py")
result.verdict       # EQUIVALENT / INEQUIVALENT / UNKNOWN
```

Key insight: No alignment problem (unlike relational Hoare logic). Failure localized to site pairs, not traces.

Question: does implementation f satisfy spec $S = C_1 \wedge \cdots \wedge C_m$?

Traditional VCGen: k paths $\times$ m conjuncts $= O(km)$ VCs. **Sheaf:** build product cover, compute Čech nerve.

- $\dim H^1 = |E| - |V| + |\pi_0| =$ redundant VCs eliminated
- H^0 components enable proof transfer (prove once, propagate)
- Layers: H^1 elim $\rightarrow$ structural $\rightarrow$ Z3 $\rightarrow$ H^0 transfer $\rightarrow$ residual Z3

```
from deppy.render.predicate_refine import SheafSpecVerifier
v = SheafSpecVerifier(z3_timeout_ms=3000)
r = v.verify(source, spec)
r.correct                    # True = proved, not just tested
r.cover.cech.h1_dimension    # redundant VCs eliminated by topology
r.eliminated_vcs             # total VCs saved
```

Key insight: VC count drops from $O(km)$ to $O(\text{independent overlaps})$. Redundancy elimination is a theorem, not a heuristic.

Key idea: every bug is a *gluing obstruction* — the bug class selects *which presheaf* fails to glue, but the pipeline is invariant.

7 obstruction families covering **20 bug classes**:

RefinementGap	div-zero, null-ptr, index-OOB, overflow, assert-fail, bounds
CarrierMismatch	type error, type confusion
TaintLeakage	information leak
ProtocolViolation	deadlock, data race, iterator protocol
ResourceLifetime	memory leak, use-after-free, double-free
ConfigWeakness	weak crypto
ExceptionEscapement	runtime error, unbound var, key error, non-termination

```
from deppy.render.bug_detect import detect_bugs
report = detect_bugs(source)
for ob in report.obstructions:
    print(ob.bug_type, ob.family, ob.conflicting_sites, ob.confidence)
```

Key insight: Adding a new bug class = defining a new presheaf + registering in `BUG_SPECS`. No new tool, same obstruction pipeline.

100-Algorithm Spec Suite

`tests/test_examples/`: 100 algorithms, each with `CORRECT`, `BUGGY`, `SPEC`, `BUG_DESC`.

Tarjan SCC, Dinic max-flow, KD-tree, Fibonacci heap, Z-algorithm, suffix array, Fenwick tree, segment tree, Dijkstra, Bellman-Ford, A*, convex hull, Rabin-Karp, Aho-Corasick, LRU cache, ...

Same `SheafSpecVerifier`, same params.

Zero per-algorithm annotation.

Čech topology adapts to each algorithm's path structure.

20-Bug-Class Detection Suite

`synthetic_bug_suite`: 20 buggy/safe pairs (40 programs).

Refinement bugs	6 classes
Carrier mismatches	2 classes
Taint leakage	1 class
Protocol violations	3 classes
Resource lifetime	3 classes
Config weakness	1 class
Exception escapement	4 classes
Total	**20 classes**

One `detect_bugs` call per program.

Each family verified on buggy (flag) and safe (clear) variants.

Five concrete benefits:

1. **Unification** — three tasks (equivalence, spec verification, bug finding), one pipeline architecture, one soundness argument
2. **Error localization** — obstructions name *site pairs*, not single points; cocycle witnesses explain *why*, not just *that*
3. **VC reduction** — Čech H^1 eliminates redundant VCs; H^0 connected components enable proof transfer
4. **Extensibility** — new bug class = new presheaf + register; Grothendieck axioms handle modular composition automatically
5. **Compositional soundness** — stability axiom $\Rightarrow$ specialization preserves analysis; transitivity axiom $\Rightarrow$ interprocedural composition for free

`github.com/???/deppy`